

P0706R0: Efficient headers for modules (or not)

Audience: Evolution Working Group
S. Davis Herring <herring@lanl.gov>
Los Alamos National Laboratory
June 18 2017

Definitions

1. A *header* is used via `#include` in the global module in a top-level context (and so does not contain a partial declaration).
2. A *source file* is a file used directly as a translation unit.
3. A *file* is a header (even if it is not really a file) or source file.
4. A *body* of a file is a contiguous portion with declarations or macro definitions but no header inclusions.
5. A *section* is a header or body (even one in a header).
6. A *section hash* is a hash of the text of a section (before preprocessing).
7. A *section name* is the name of a header or a section hash of a body.
8. The *contents* of a section are its declarations, implicit instantiations, and (net) macros.
9. The *names used* by a section are the names encountered in it either during or after preprocessing.
10. The *cache* of a module, file, or section is its serialized form.
11. A *database* is a set of caches (with indices to be specified).
12. A *cache hash* is a (partial) key for the database derived from the corresponding value, long enough to be assumed collision-free.

Issues

1. Many if not all of the definitions in a header must appear in the cache of every module that includes it to support template instantiation (both templates in the header and instantiation with types derived from the header).
 1. One implementation of N4637 serializes all the referenced classes from a header.
 2. Non-member functions would also have to be serialized to support ADL (the ill-formed example from P0582R0).
2. A change in a header forces recompilation of all translation units that include it, even if they are not affected by the change.
 1. For a module, the new version must appear in its cache.
 2. For a traditional compile, nothing may be assumed about the effects.
3. While it is often an ODR violation, many headers in practice are noncommutative, limiting the utility of traditional precompiled headers.
 1. We want to "support" these in as close a fashion as possible to traditional separate compilation behavior.
 2. We want to detect violations so as to (gradually) remedy them.

Goals

1. Store each declaration in one cache.
 1. Support fast lookup in all modules (to fix P0582R0's example).
 2. "Allow" ODR-violating definitions in different caches.

3. Warn about or reject such violations.
2. Avoid reparsing of dependent files on insignificant changes.
3. Take advantage of existing organization into headers.
4. Improve compilation performance even in the absence of any modules (conversely, in the presence of macros).
5. Handle include guards without repeated preprocessing.

Non-strategies

1. In the absence of trickery (*e.g.*, macro-based template emulation), it would be sufficient to store a cache for each header (as if it were a module that exported all its contents): this is just precompiled headers.
2. If prefixes of certain long sequences of headers were frequently included at the beginning of a translation unit (but others might appear with meaningfully different orders), we could use the sequence of header names as a database key.
3. If changes to a header were either irrelevant to all clients (*e.g.*, comments) or relevant to all clients (*e.g.*, a header containing a single class), we could use (a hash of) the contents of antecedent headers and the (next) header's name as a database key.

Strategy

1. Store in a database one or more caches for each section, containing the section's contents and the interpretation of its names used (as a macro or from name lookup), as well as a section hash (not included in constructing the cache hash) to detect alteration.
2. Store in a database a cache for each file (indexed simply by the file's name), containing a mapping from section names to section cache hashes. (For a source file, the object code might reasonably also be included so as to provide the basic functionality of `make(1)`.)
3. When recompiling a file, the caches identified by the cached mapping are checked. If the section has not changed textually and (despite any changes earlier in the recompilation) its names used have the same interpretations when looked up in the new compilation context as they have in the cache, then the cached contents are used. (Note that this can occur even after encountering semantically significant changes if they do not affect the interpretation of the section in question.) Otherwise, the section is recompiled and recached -- note that if the textual changes are insignificant, the section contents will match those in the cache (but its section hash is updated) and the "no changes" state of compilation persists.
4. Since bodies do not have names, changed bodies will simply not be found in the cache at all. Those with insignificant changes will nonetheless allow the "no changes" to persist.
5. `#include` directives that do not qualify as headers are not cached separately, but treated as the textual inclusion that they are.
6. Any of these lookups may be augmented with the set of compiler options that may affect program behavior, so as to maintain different versions simultaneously.
7. The database must be garbage-collected, especially for bodies which (lacking names) cannot be replaced when modified.
8. Module interface units can be treated in a fashion very similar to that for other source files, but with the export status of each declaration included so that other translation units that import the module can use the (appropriate subset of the) cache.

Notes

1. When a header is "skipped" because of include guards, its only name used will be the guard macro, so that recompiling will succeed at reusing the cache regardless of changed contents.
2. A module can but need not be automatically (re)compiled and cached when an import of it is encountered.

3. ODR violations can easily be detected at "link time" by consulting the union of the caches for the various translation units (making sure to include the definitions, or at least hashes of them, in the caches for this purpose).